

SKILL (OSSP)

SESSION 1

1.To Install Linux VM, Configure GCC, Setup Git Repository, Create Project Structure, Understand Shell Architecture, Build Initial Makefile.

```
koumudi@Koumudi:~/OSSP$ tree
.
├── LICENSE
├── Makefile
├── PRACTICAL
│   ├── Session_1
│   ├── Session_2
│   └── Session_3
├── README.md
├── SKILL
│   ├── Session_1
│   └── Session_2
├── assets
│   └── architecture.png
├── bin
├── docs
│   ├── design.md
│   └── report.pdf
├── include
│   ├── builtin.h
│   ├── executor.h
│   ├── parser.h
│   └── shell.h
├── obj
├── scripts
│   └── run.sh
└── src
    ├── builtin.c
    ├── executor.c
    └── main.c
```

```

├── shell.h
├── obj
├── scripts
│   └── run.sh
├── src
│   ├── builtin.c
│   ├── executor.c
│   ├── main.c
│   ├── parser.c
│   └── shell.c
└── tests
    ├── test_executor.c
    └── test_parser.c

```

16 directories, 18 files
koumudi@Koumudi:~/OSSP\$ |

```

GNU nano 7.2 Makefile *
CC = gcc
CFLAGS = -Wall -Iinclude

SRC = src/main.c src/shell.c src/parser.c src/executor.c src/builtin.c
OBJ = $(SRC:src/%.c=obj/%.o)

TARGET = bin/my_shell

all: $(TARGET)

$(TARGET): $(OBJ)
$(CC) $(OBJ) -o $(TARGET)

obj/%.o: src/%.c
$(CC) $(CFLAGS) -c $< -o $@

clean:
rm -f obj/*.o $(TARGET)

```

```

koumudi@Koumudi:~/OSSP$ nano Makefile
koumudi@Koumudi:~/OSSP$ cat Makefile
CC = gcc
CFLAGS = -Wall -Iinclude

SRC = src/main.c src/shell.c src/parser.c src/executor.c src/builtin.c
OBJ = $(SRC:src/%.c=obj/%.o)

TARGET = bin/my_shell

all: $(TARGET)

$(TARGET): $(OBJ)
$(CC) $(OBJ) -o $(TARGET)

obj/%.o: src/%.c
$(CC) $(CFLAGS) -c $< -o $@

clean:
rm -f obj/*.o $(TARGET)
koumudi@Koumudi:~/OSSP$ |

```

2. To Analyze process abstraction, execute fork(), understand exec() family, analyze parent-child relationships, inspect process tree, practice system call tracing.

```
koumudi@Koumudi:~$ cd ~/OSSP/SKILL/Session_1
koumudi@Koumudi:~/OSSP/SKILL/Session_1$ pwd
/home/koumudi/OSSP/SKILL/Session_1
koumudi@Koumudi:~/OSSP/SKILL/Session_1$ gcc --version
gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
Copyright (C) 2023 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

koumudi@Koumudi:~/OSSP/SKILL/Session_1$ git --version
git version 2.43.0
koumudi@Koumudi:~/OSSP/SKILL/Session_1$ cd ~/OSSP
git status
On branch master

No commits yet

nothing to commit (create/copy files and use "git add" to track)
koumudi@Koumudi:~/OSSP$ mkdir -p src include obj bin docs tests scripts assets
koumudi@Koumudi:~/OSSP$ touch .gitignore README.md LICENSE Makefile
touch src/main.c src/shell.c src/parser.c src/executor.c src/builtin.c
touch include/shell.h include/parser.h include/executor.h include/builtin.h
touch docs/design.md docs/report.pdf
touch tests/test_parser.c tests/test_executor.c
touch scripts/run.sh
touch assets/architecture.png
koumudi@Koumudi:~/OSSP$ tree
```

```
koumudi@Koumudi: ~/OSSP/ × + ▾
GNU nano 7.2
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main()
{
    pid_t pid;

    printf("Parent process started.\n");
    printf("Parent PID = %d\n", getpid());
    printf("Parent PPID = %d\n", getppid());

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        printf("\n--- CHILD PROCESS ---\n");
        printf("Child PID = %d\n", getpid());
        printf("Child PPID = %d\n", getppid());
```

```

koumudi@Koumudi:~/OSSP$ cd ~/OSSP/SKILL/Session_1
koumudi@Koumudi:~/OSSP/SKILL/Session_1$ nano process.c
koumudi@Koumudi:~/OSSP/SKILL/Session_1$ gcc process.c -o process
koumudi@Koumudi:~/OSSP/SKILL/Session_1$ ./process
Parent process started.
Parent PID = 2183
Parent PPID = 417

--- PARENT PROCESS ---
Parent PID = 2183
Parent waiting for child...

--- CHILD PROCESS ---
Child PID = 2184
Child PPID = 2183
Child executing 'ls' using execlp()...
process process.c
Child execution completed.
Parent process continuing.
koumudi@Koumudi:~/OSSP/SKILL/Session_1$ |
```


2. To Capture Keyboard Input, Handle Backspace, Process Enter Key, Manage Input Buffer, Support Multi-Character Commands, Test User Interaction.

```
koumudi@Koumudi:~/OSSP/SKILL/Session_2$ gcc shell_loop.c -o shell_loop
koumudi@Koumudi:~/OSSP/SKILL/Session_2$ ./shell_loop
my_shell> hello
You entered: hello
my_shell> test
You entered: test
my_shell>
You entered:
my_shell> exit
Exiting shell...
koumudi@Koumudi:~/OSSP/SKILL/Session_2$ nano keyboard_input.c
koumudi@Koumudi:~/OSSP/SKILL/Session_2$ gcc keyboard_input.c -o keyboard_input
koumudi@Koumudi:~/OSSP/SKILL/Session_2$ ./keyboard_input
my_shell> hello
You entered: hello
my_shell> helloo
You entered: helloo
my_shell> hello
You entered: hello
my_shell> exit
You entered: exit
Exiting shell...
koumudi@Koumudi:~/OSSP/SKILL/Session_2$ |
```